Roteiro de Atividade Prática
Nome: Arthur Saraiva de França 3°B

Projetando o DevPipeline — Sistema multiagente de desenvolvimento

Parte 1 — O agente monolítico (o problema)
A StartCode S.A. construiu o DevAgent, um único agente de IA responsável por todo o ciclo de desenvolvimento de software: receber requisitos, escrever código, revisar bugs, criar testes automatizados, gerar documentação e fazer o deploy em produção. O agente é extraordinariamente versátil, mas os engenheiros relatam os seguintes problemas após 3 meses de uso: qualidade inconsistente, contexto excessivo, impossibilidade de paralelizar e rastreabilidade zero.

1.1 Para cada problema listado, identifique qual limitação do modelo monolítico está presente. Como a colaboração entre múltiplos agentes resolveria o problema?
Resposta: A qualidade inconsistente revela o viés de autoavaliação, que é resolvido delegando a revisão a um agente auditor independente. O contexto excessivo mostra a limitação de memória longa, mitigada pela divisão de tarefas em agentes focados que recebem apenas dados essenciais. A impossibilidade de paralelizar decorre do fluxo estritamente sequencial, solucionado pela execução síncrona de agentes independentes. A rastreabilidade zero expõe a falta de granularidade do monólito, corrigida pelo isolamento das ações em etapas claras para cada agente.

1.2 O DevAgent precisa ”ser bom em tudo“: requisitos, código, testes, deploy. Por que a especialização de agentes tende a produzir resultados superiores à generalização em um único agente?
Resposta: A especialização reduz a complexidade do prompt e otimiza as instruções para uma única habilidade técnica específica. Ao focar em apenas uma tarefa, o agente reduz desperdício de tokens, evita confusão de papéis e apresenta maior consistência na execução de atividades complexas.

Parte 2 — Projetando a colaboração entre agentes
Agora você não vai apenas dividir o sistema em agentes. Você vai projetar como eles trabalham juntos.

2.1 Descreva o fluxo do DevPipeline, indicando a ordem de atuação dos agentes.
Resposta:
Etapa 1 - Agente de Requisitos
Etapa 2 - Agente de Código
Etapa 3 - Agente de Testes e Revisão
Etapa 4 - Agente de Deploy e Documentação

2.2 Comunicação entre agentes (handoff). Para cada etapa, explique o que o agente recebe, o que ele produz e o que ele entrega ao próximo agente.
Resposta: O Agente de Requisitos recebe as demandas do cliente, produz a especificação técnica detalhada e a entrega para o Agente de Código. O Agente de Código recebe a especificação técnica, produz o código do software e o entrega para o Agente de Testes e Revisão. O Agente de Testes e Revisão recebe o código construído, produz os relatórios de validação com correções de bugs e entrega o código aprovado para o Agente de Deploy. O Agente de Deploy recebe o código validado, produz o software em produção com registros de infraestrutura e finaliza o ciclo de entrega.

2.3 — Indique quais etapas poderiam acontecer ao mesmo tempo (em paralelo) e explique por quê.
Resposta: A criação da documentação técnica e a escrita de testes automatizados podem ocorrer ao mesmo tempo. Isso é possível porque ambas as etapas dependem apenas do código finalizado produzido na etapa anterior, sem que uma precise do resultado da outra para avançar.

2.4 — Explique como o sistema pode identificar em qual etapa ocorreu um erro.
Resposta: O sistema monitora as entradas e saídas de cada etapa por meio de logs estruturados e validações exclusivas em cada handoff. Se um agente falhar em gerar a saída esperada ou se a validação automática apontar erros nas entregas individuais, o sistema interrompe o pipeline e isola o problema na etapa exata do agente responsável.